FINAL PROJECT REPORT

Script-Controlled ACL

Restrict Record Access Based on Field Value

Project Platform: ServiceNow

Project Version: 1.0

Team ID: SWTID-2026-8089

Report Date: 03 October 2026

1. INTRODUCTION

1.1 Project Overview

The project, “Script-Controlled ACL,” implements record-level security in ServiceNow. The implementation uses a custom table, custom roles and script-controlled Access Control Lists (ACLs) so that users can view, create, edit and delete records only when they hold the required role and the record satisfies the specified condition. In this scenario, only users holding the bb1 role can view records where the Branch field is set to EEE, while administrators retain full access.

1.2 Purpose

The purpose of the project is to establish a repeatable ServiceNow security configuration that restricts record access based on a field value. The intended process connects user and role creation, a custom table, and READ, CREATE, WRITE and DELETE ACLs in one controlled access model.

2. IDEATION PHASE

2.1 Problem Statement

The project addresses the risk of sensitive student and institution records being exposed to users who should not see them. Role-only security cannot restrict visibility by a field value such as Branch, so users could view, modify or delete records of other branches. A script-controlled ACL is required so that access is decided by evaluating the user’s roles together with the record’s field values.

2.2 Empathy Map Canvas

The completed Empathy Map Canvas from Phase 1 should be inserted or referenced here.

Reference artifact: Empathy Map Canvas.docx / Empathy Map Canvas.docx.pdf

2.3 Brainstorming

The completed brainstorming and idea-prioritization artifact from Phase 1 should be inserted or referenced here.

Reference artifacts: Brainstorming- Idea Generation- Prioritization Template.docx and Brainstorming Template.pdf

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The team’s completed Customer Journey Map from Phase 2 should be inserted or referenced here.

Reference artifact: Customer Journey Map - Example.pdf

3.2 Solution Requirement

The solution requires a custom Institution Details table (u_institution_details) to be protected by script-controlled ACLs. A user with the bb1 role must be able to read only records where Branch is EEE; a user without the role must see no records; and an administrator must see all records. Separate roles must control creating (bb2), editing (bb3) and deleting (bb4) records.

3.3 Data Flow Diagram

The team's completed Data Flow Diagram should be inserted or referenced here.

Reference artifact: Data Flow Diagrams and User Stories.docx / PDF

3.4 Technology Stack

Technology / Component	Purpose
ServiceNow Platform	Used to implement and execute the record-level security configuration.
User Administration (Users & Roles)	Provides the EEE User and the custom roles bb1, bb2, bb3 and bb4.
Custom Table (u_institution_details)	Stores the Institution Details records, including the Branch field (ECE, EEE, CSE).
Access Control (ACL)	Enforces READ, CREATE, WRITE and DELETE permissions on the table.
Advanced ACL Script (gs.hasRole)	Evaluates the user's roles in script to allow or deny read access.
security_admin role	Elevated role required to create and modify ACLs.
Impersonation	Used to verify access as the EEE User, a user without roles and the administrator.

4. PROJECT DESIGN

4.1 Problem Solution Fit

The team's completed Problem-Solution Fit artifact from Phase 3 should be inserted or referenced here.

Reference artifacts: Problem - Solution Fit Template v1.docx / PDF and related Phase 3 files.

4.2 Proposed Solution

The proposed solution is a ServiceNow security configuration built on the custom table Institution Details. The EEE User is assigned the custom roles bb1, bb2, bb3 and bb4. A READ ACL requires the bb1 role, applies the data condition "Branch is EEE" and uses an advanced script based on gs.hasRole() so that administrators retain full access. Separate CREATE (bb2), WRITE (bb3) and DELETE (bb4) ACLs control who can add, edit and remove records.

4.3 Solution Architecture

The implemented logical flow is:

Administrator → Users, Roles & Custom Table → ACLs (Read / Create / Write / Delete) → User opens Institution Details list → ACL checks role and Branch condition → Allow or Deny

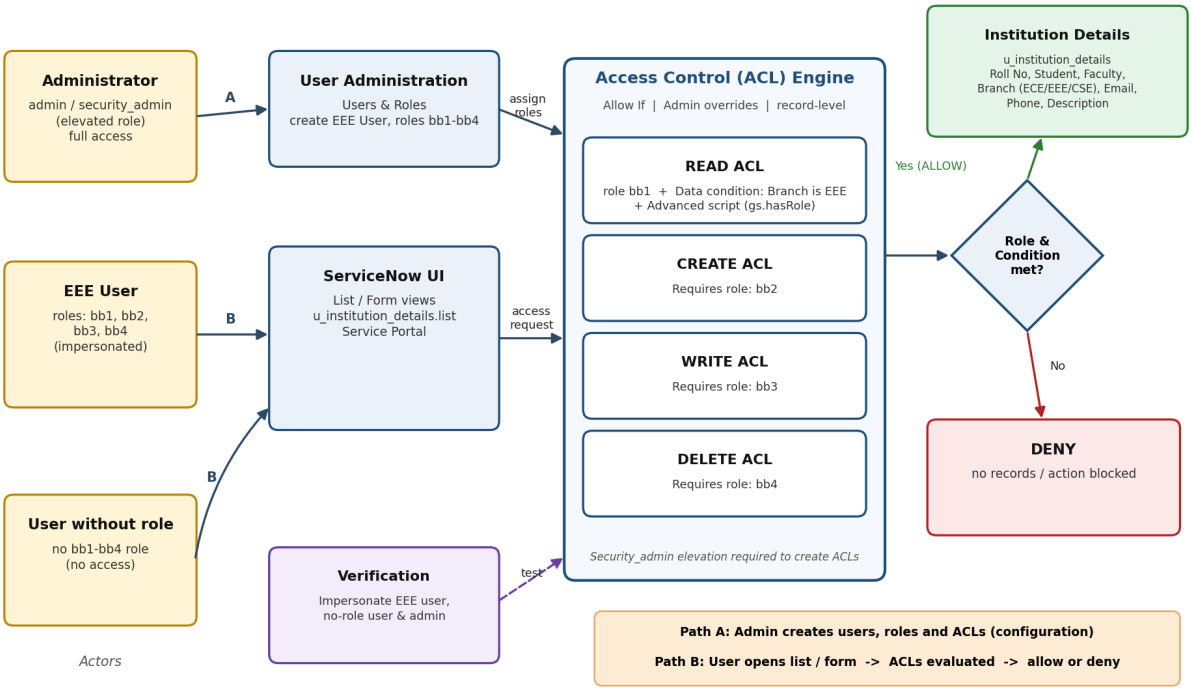


Figure 1: Solution architecture of the Script-Controlled ACL application

The completed Solution Architecture artifact from Phase 3 should also be inserted or referenced here.

Reference artifacts: Solution Architecture.docx / PDF

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The team's completed Project Planning Template and planning logic documents from Phase 4 should be inserted or referenced here.

Reference artifacts: Project Planning Template.docx / PDF and Planning logic.docx

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Functional and execution testing was documented for the implemented ACL configuration. The tested paths covered read access for the bb1 role, denial of access for a user without roles, full access for the administrator, and the create, write and delete operations controlled by bb2, bb3 and bb4.

Test ID	Scenario	Result	Measurement Status
PT-001	EEE User (bb1) views Institution Details list	PASS	Functional execution verified
PT-002	User without roles opens the list	PASS	Functional execution verified
PT-003	Administrator views all records	PASS	Functional execution verified

Test ID	Scenario	Result	Measurement Status
PT-004	Create access (bb2) – New button available	PASS	Functional execution verified
PT-005	Write access (bb3) – record edited	PASS	Functional execution verified
PT-006	Delete access (bb4) – record deleted	PASS	Functional execution verified
PT-007	Measured response-time benchmark	NOT MEASURED	No quantitative benchmark recorded

No response-time, throughput, or concurrent-user benchmark is claimed in this report because those quantitative measurements were not recorded in the supplied project evidence.

7. RESULTS

7.1 Output Screenshots

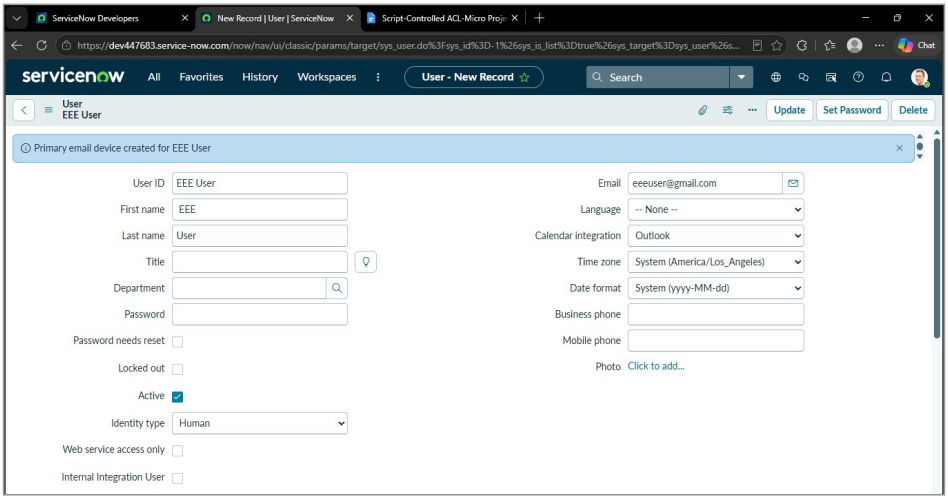


Figure 2: EEE User created in User Administration

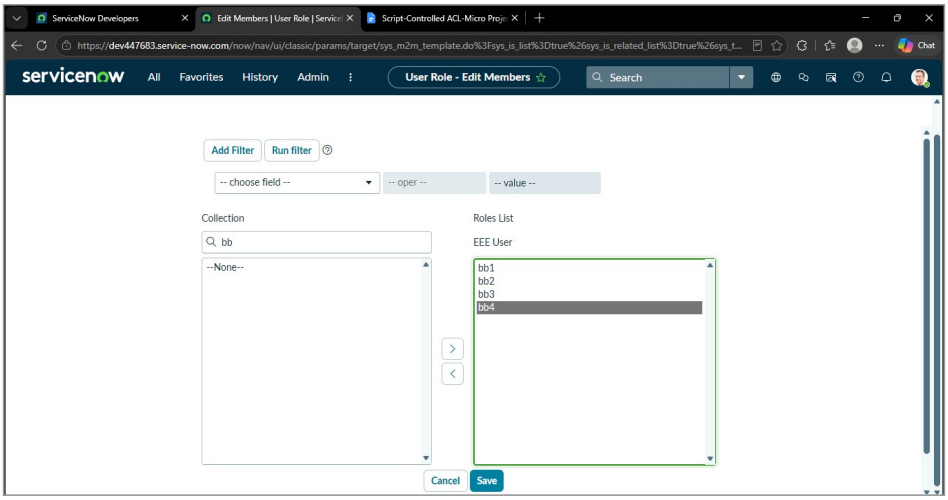


Figure 3: Custom roles bb1–bb4 assigned to the EEE User

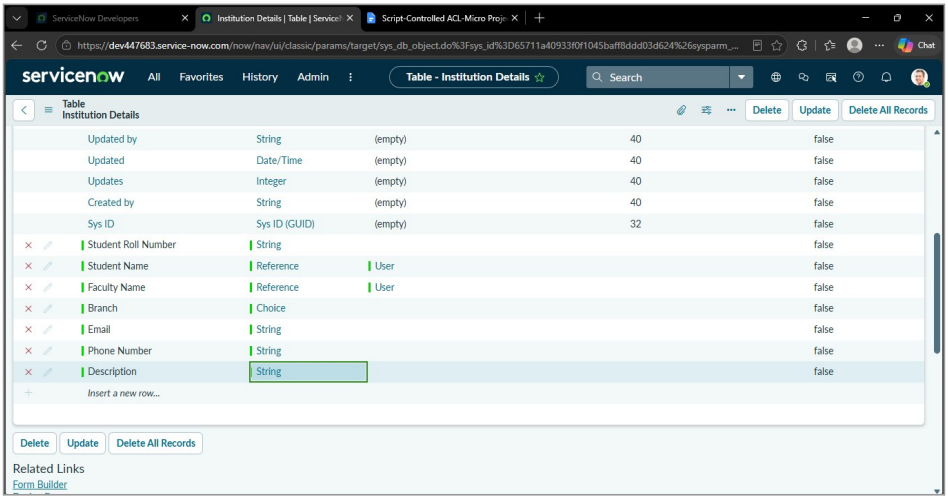


Figure 4: Institution Details table (u_institution_details) with required fields

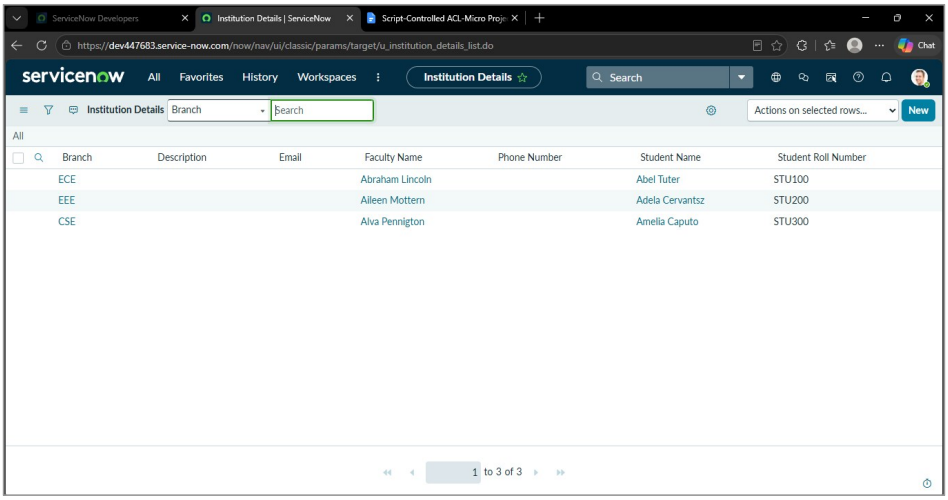


Figure 5: Sample records with Branch values ECE, EEE and CSE (admin view)

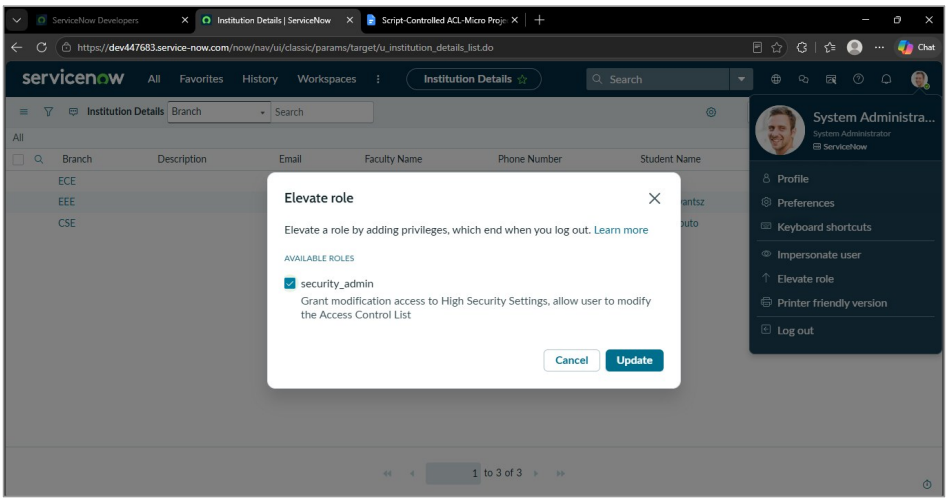


Figure 6: Elevating the security_admin role to create ACLs

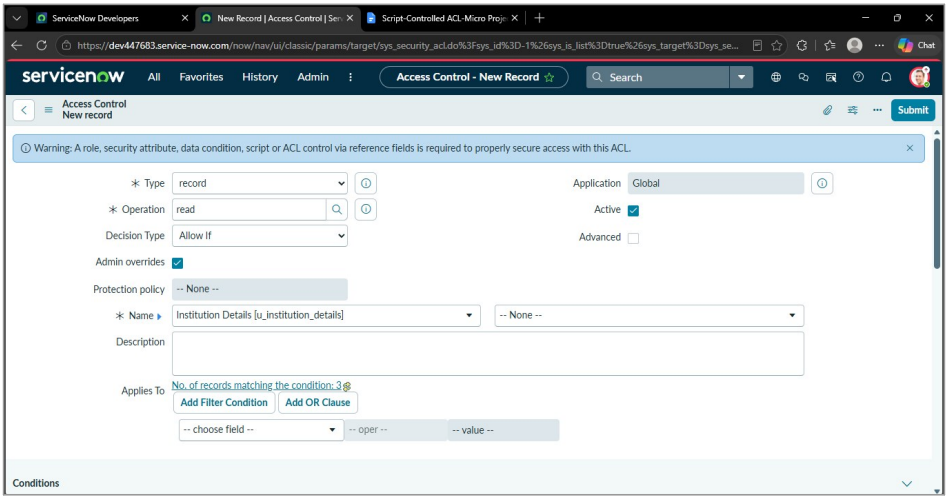


Figure 7: Read ACL – Type record, Operation read, Name u_institution_details

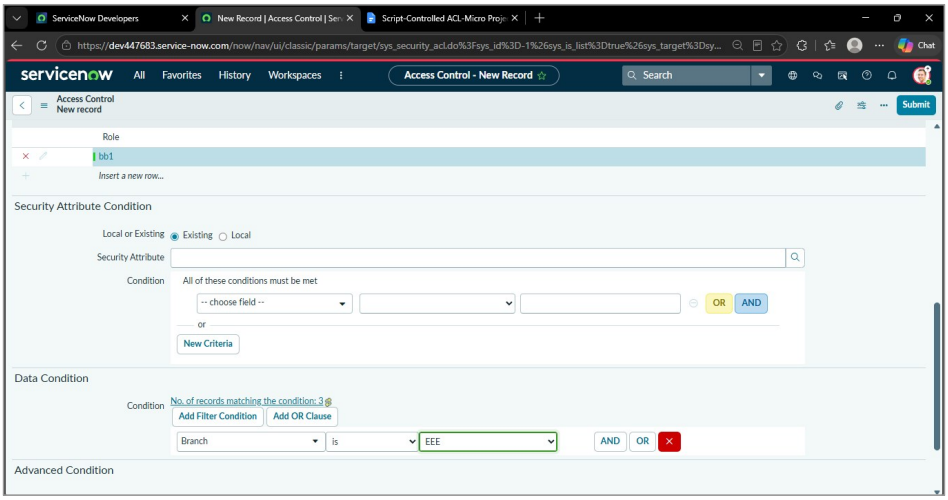


Figure 8: Read ACL – Requires role bb1 and Data condition Branch is EEE

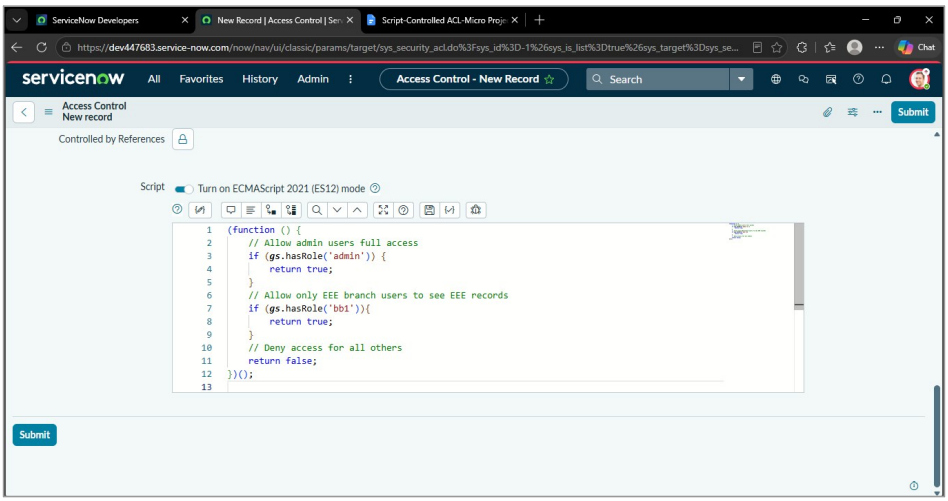


Figure 9: Read ACL – Advanced script using gs.hasRole()

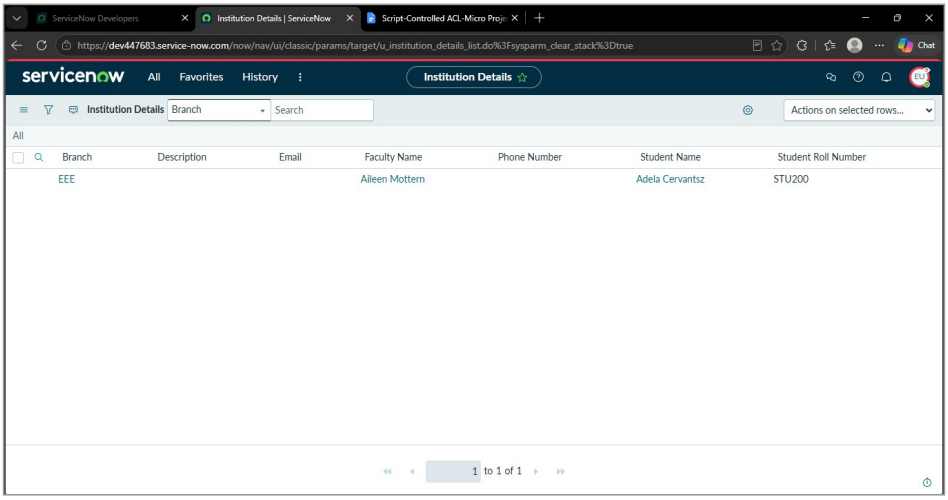


Figure 10: EEE User (bb1) views only EEE branch records

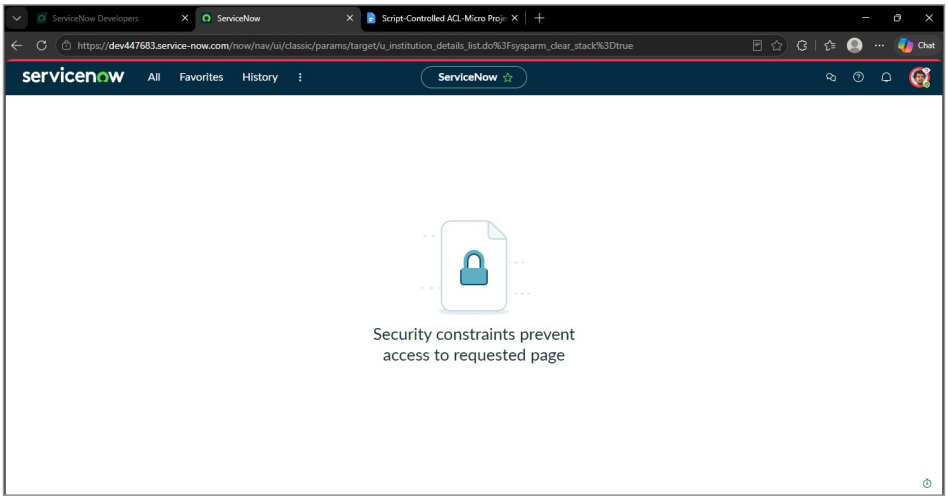


Figure 11: User without the roles – security constraints prevent access

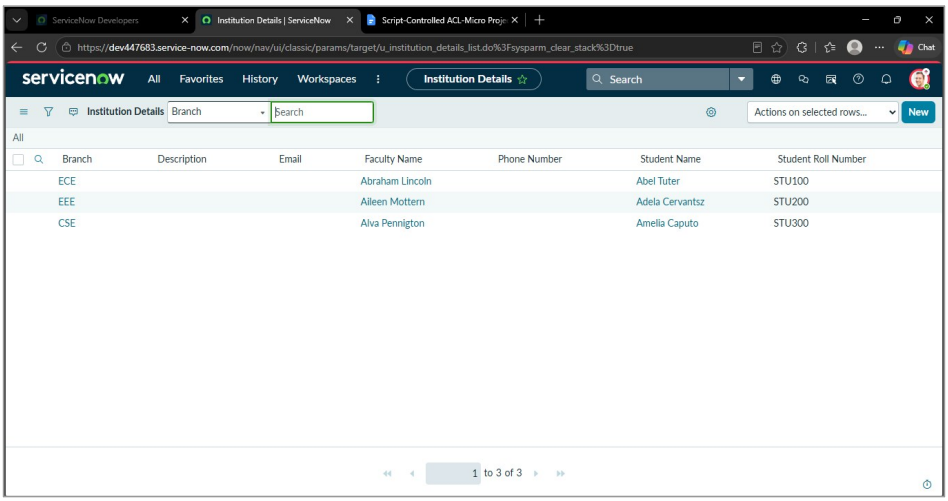


Figure 12: Administrator views all records regardless of branch

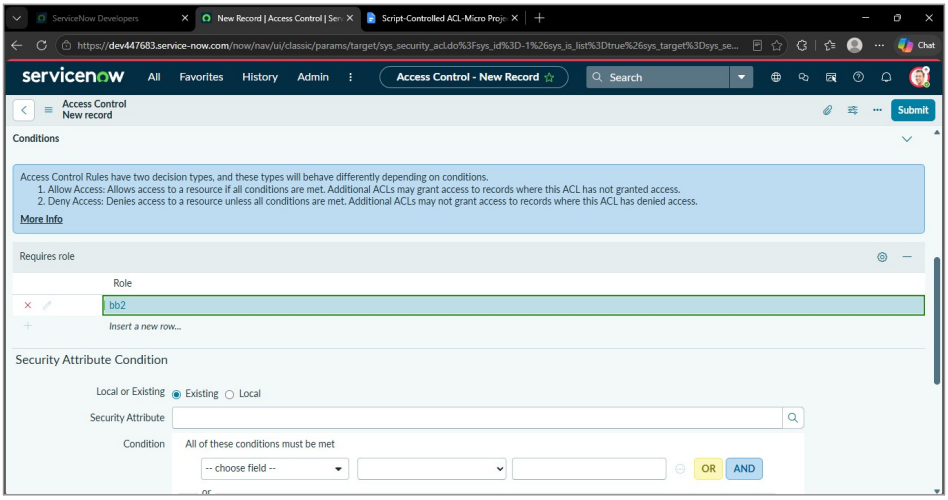


Figure 13: Create ACL – Requires role bb2

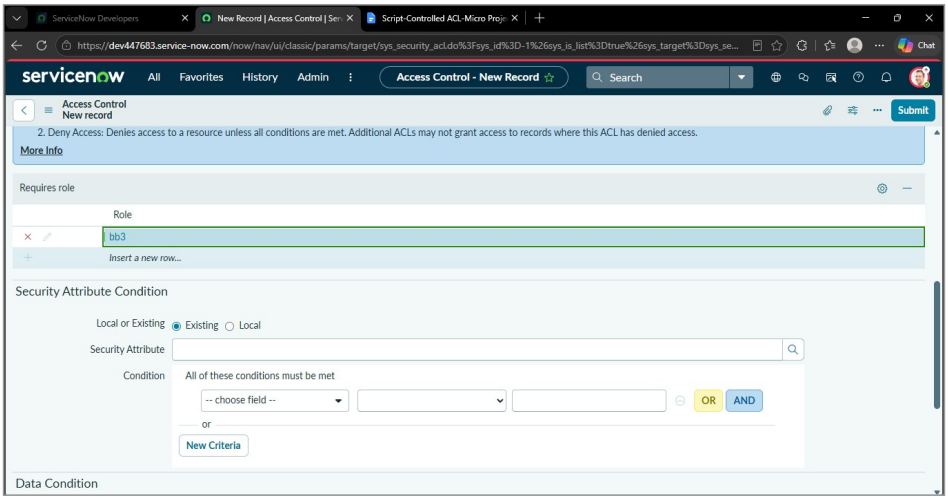


Figure 14: Write ACL – Requires role bb3

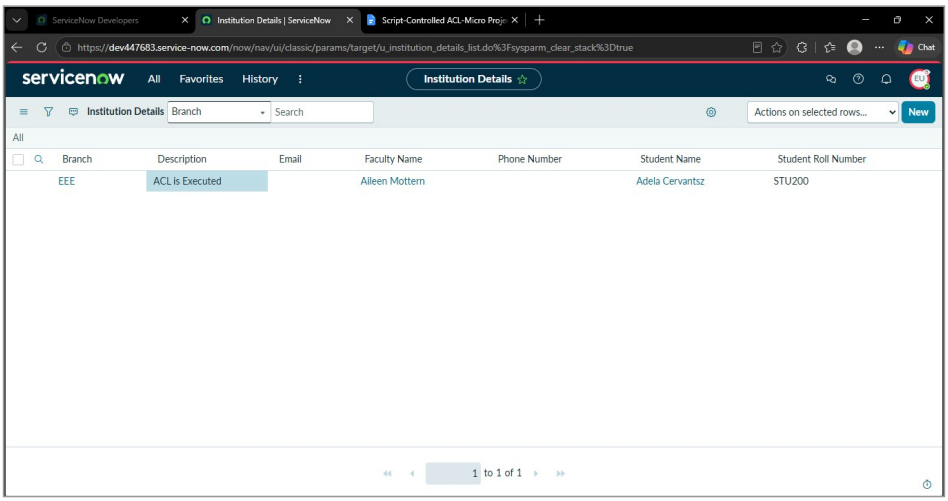


Figure 15: Record edited by a user with write access (Description: ACL is Executed)

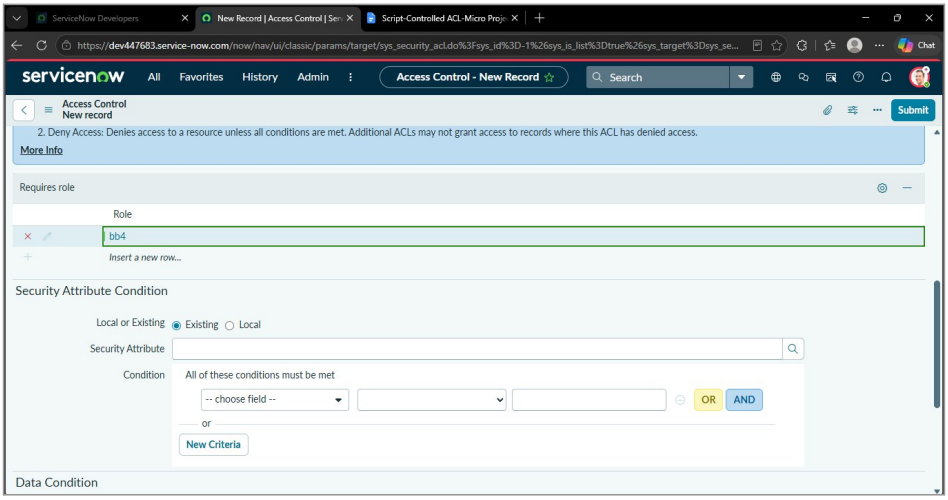


Figure 16: Delete ACL – Requires role bb4

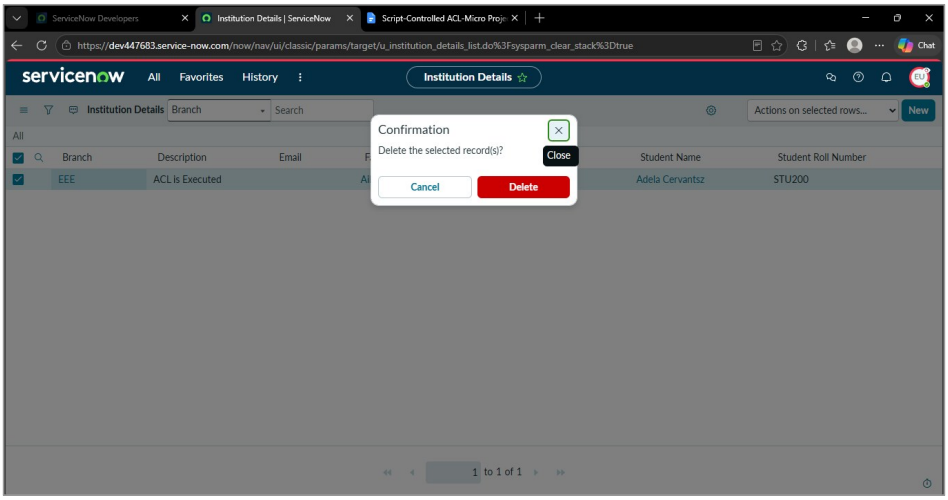


Figure 17: Delete confirmation for the selected EEE record

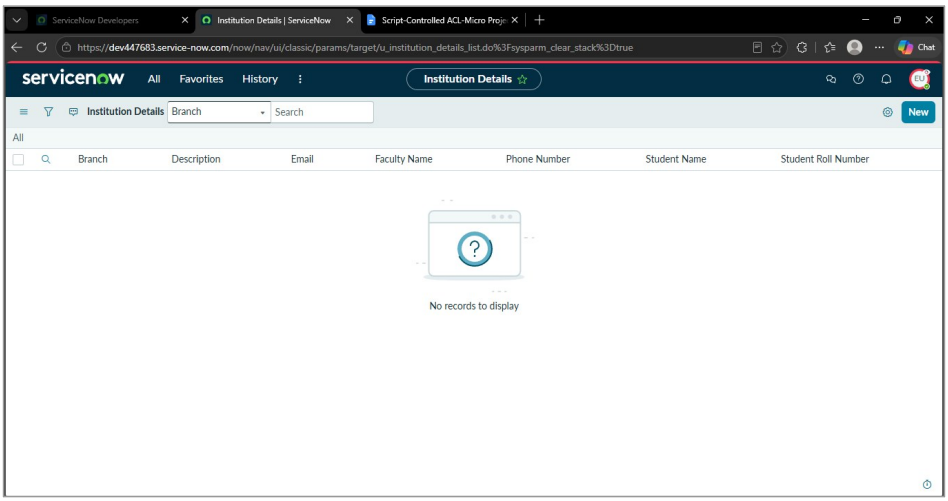


Figure 18: List after deletion – record removed

8. ADVANTAGES & DISADVANTAGES

8.1 Advantages

- Restricts record visibility based on a field value, not only on roles.
- Provides separate, granular control over read, create, write and delete operations.
- Keeps full access for administrators through the ACL script.
- Uses ServiceNow-native ACL, role and table capabilities with no external tools.
- Creates a repeatable security pattern that can be demonstrated and tested through impersonation.

8.2 Disadvantages / Limitations

- The current implementation is focused on the EEE branch of the Institution Details table.
- Quantitative performance benchmarks were not recorded.
- The process depends on the roles, ACL records and data conditions being correctly maintained.
- The report does not claim field-level security or integration with external systems unless added later.

9. CONCLUSION

The project implements a ServiceNow-based record-level security solution using script-controlled ACLs. The configured READ, CREATE, WRITE and DELETE ACLs evaluate user roles together with the Branch field value, so that bb1 users see only EEE branch records, users without roles see none, and administrators see all. The documented execution path demonstrates how ACLs protect sensitive data from unauthorized viewing, modification, creation and deletion.

10. FUTURE SCOPE

- Extend the ACLs to the ECE and CSE branches with dedicated roles and data conditions.
- Add field-level ACLs to protect sensitive fields such as Email and Phone Number.
- Use groups instead of individual roles to simplify access administration.
- Apply the same ACL pattern to Service Portal widgets and additional custom tables.
- Capture quantitative performance metrics such as list load time under ACL evaluation.
- Add reporting or dashboards for access audits and role assignments.

11. APPENDIX

11.1 Source Code

The project is primarily configured within the ServiceNow platform. The advanced script used in the READ ACL is shown below.

```
(function () {
  // Allow admin users full access
  if (gs.hasRole('admin')) {
    return true;
  }
  // Allow only EEE branch users to see EEE records
  if (gs.hasRole('bb1')) {
    return true;
  }
  // Deny access for all others
  return false;
})();
```

11.2 Dataset Link

Not applicable. The implemented solution does not depend on a machine-learning dataset.

11.3 GitHub & Project Demo Link

GitHub Repository: (to be added by the team)

Project Demo Video: (to be added by the team)

SkillWallet Project Link: (to be added by the team)

12. PROJECT PHASE ARTIFACT CHECKLIST

Phase	Artifact / Deliverable	Status
1. Ideation	Problem statement, Empathy Map, Brainstorming	Completed / team artifact to attach
2. Requirement Analysis	Customer Journey, Solution Requirements, DFD, Technology Stack	Completed / team artifact to attach
3. Project Design	Problem-Solution Fit, Proposed Solution, Solution Architecture	Completed / team artifact to attach
4. Project Planning	Planning Logic, Project Planning	Completed / team artifact to attach
5. Project Development	UAT and Performance Testing	Completed
6. Project Documentation	FSD Documentation and Final Report	Final Report completed
7. Demo Video	Project demonstration	To be finalized / link to add